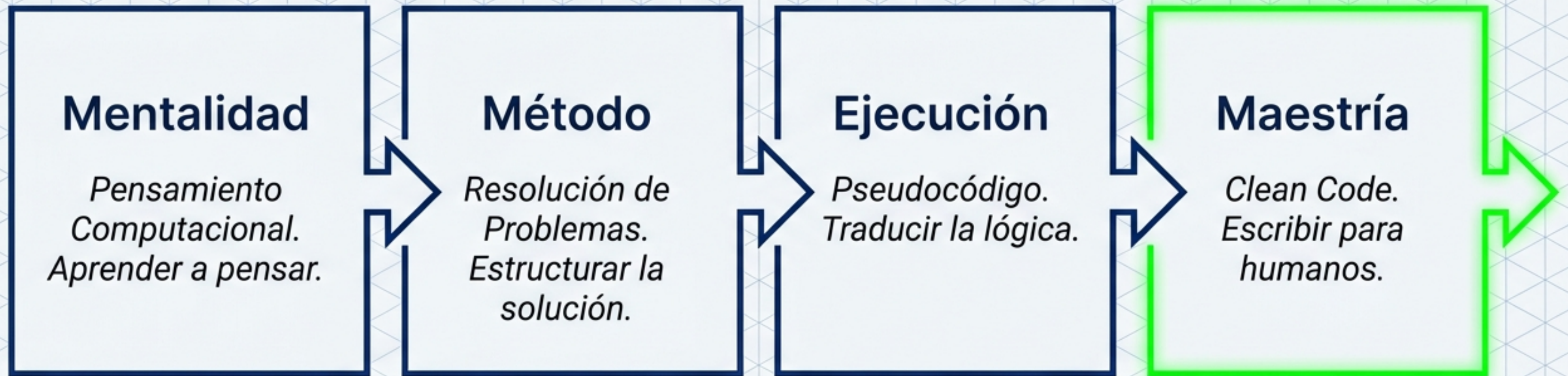


De la Lógica al Código Limpio

Fundamentos de Pensamiento Computacional,
Algoritmos y Buenas Prácticas

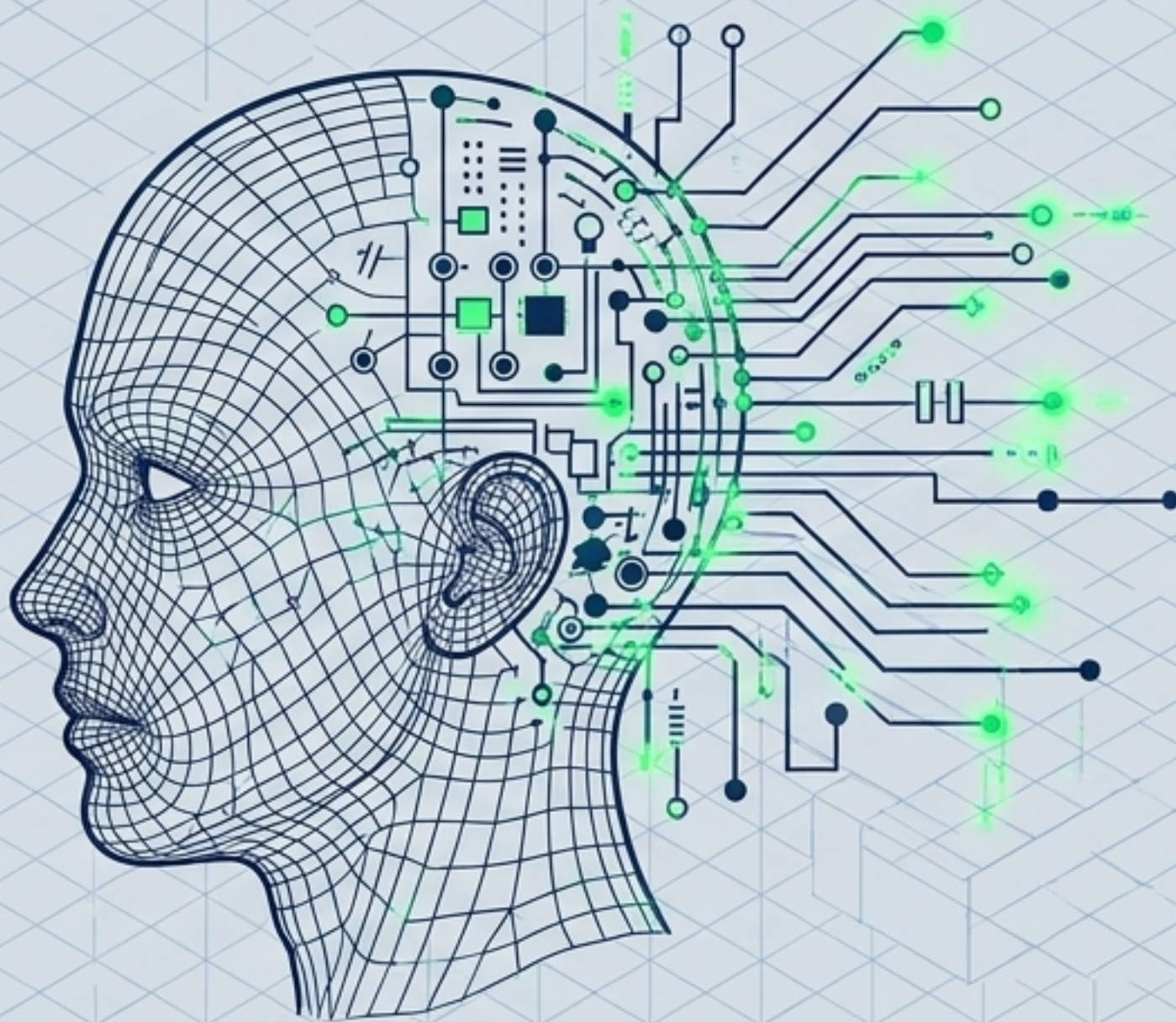
Una guía maestra para el diseño, estructura y
artesanía del software profesional.

```
function optimizeAlgorithm(data) {  
  // ...  
  let result = process(data);  
  return result;  
}  
  
class DataProcessor {  
  def __init__(self, stream):  
    self.stream = stream;  
  def process(self):  
    self.stream = stream;  
    const CLEAN_CODE_PRINCIPLES =  
    ['Readability', 'Simplicity', 'Modularity'];  
    if (condition.isValid()) {  
      executePathway(A);  
    } else {  
      executePathway(B);  
    }  
    if (condition.isValid()) {  
      executePathway(C);  
    }  
  }  
}  
  
const DP = new DataProcessor(stream);  
DP.process();  
return DP;
```

La Regla del 80/20

Escribir software es **80% pensar y estructurar**, y **20% tipear código**. El verdadero trabajo ocurre antes de tocar el teclado.



“El pensamiento computacional implica los procesos de pensamiento involucrados en la formulación de un problema y la expresión de sus soluciones de tal manera que un agente (humano o máquina) pueda llevarlas a cabo eficazmente.”

— Jeannette M. Wing, 2006

No es saber programar; es saber mirar el mundo a través de la lógica de sistemas, abstracción y automatización.



Descomposición

Desarticular un problema complejo en pequeñas series manejables ("Divide y vencerás").



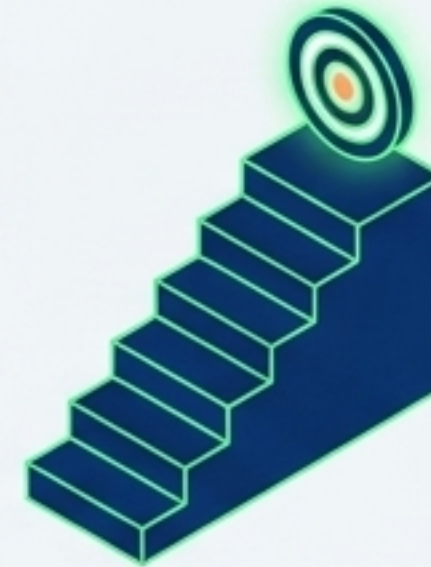
Reconocimiento de Patrones

Identificar similitudes y tendencias para aplicar soluciones conocidas a nuevos retos.



Abstracción

Omitir la información irrelevante y enfocarse estrictamente en las variables necesarias.



Algoritmos

Diseñar la secuencia lógica y ordenada de pasos para automatizar la solución.



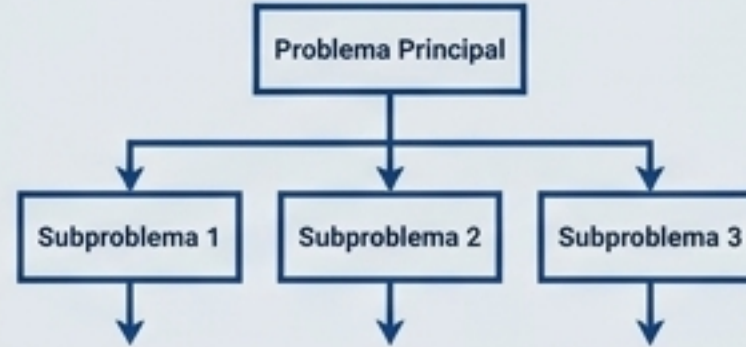
El diseño previo es obligatorio.
Sin algoritmo, no puede existir un programa.

Entrada (Datos)

Información inicial.
Requiere abstracción
para identificar solo las
variables relevantes al
problema.



Proceso (Lógica)



Operaciones, fórmulas y
estructuras de control
(condicionales y bucles)
necesarias para
transformar los datos.



Salida (Resultados)

La solución final o
respuesta dada por el
algoritmo al usuario.

LÓGICA HUMANA

Lenguaje Natural - Alta ambigüedad.

Preguntarle la edad al usuario y decirle si es mayor de edad.

EL PUENTE (PSeInt)

Pseudocódigo en español. Sintaxis flexible pero estructurada. Elimina la barrera del idioma y los detalles de memoria.

```
Leer edad  
Si edad >= 18 Entonces  
    Escribir "Mayor de edad"
```

LÓGICA DE MÁQUINA

Lenguaje C++. Sintaxis estricta, tipos de datos rígidos, manejo de memoria y compilación real.

```
cin >> edad;  
if (edad >= 18) {  
    cout << "Mayor de edad";  
}
```


Cabecera y Cierre

Delimita el inicio y fin del algoritmo.



Buenas Prácticas

Obligatorio declarar y tipificar TODAS las variables al inicio. Previene errores de ejecución.

FinProceso

```
Proceso CalculadoraBase
// Zona 1: Declaraciones
Definir num1, num2, resultado Como Real
// Zona 2: Entradas
Escribir "Ingrese números:"
Leer num1, num2
// Zona 3: Procesamiento
resultado <- num1 + num2
// Zona 4: Salida
Escribir "Total: ", resultado
FinProceso
```

Bloque E-P-S

La implementación directa del paradigma Entrada-Proceso-Salida.

Tipo de Dato	Definición	Operadores Clave
Numérico (Entero / Real)	Valores matemáticos. Enteros o decimales (separados por punto).	Algebraicos: +, -, *, /, MOD, %
Lógico (Booleano)	Valores de verdad binarios (VERDADERO o FALSO).	Lógicos: Y, O, NO
Carácter (Texto)	Letras o cadenas encerradas en comillas dobles o simples ("hola").	Relacionales: =, <>, >, <, >=, <=

INTERACCIÓN USUARIO-SISTEMA



Asignación (<- o =)

Guarda un valor procesado en la memoria RAM.

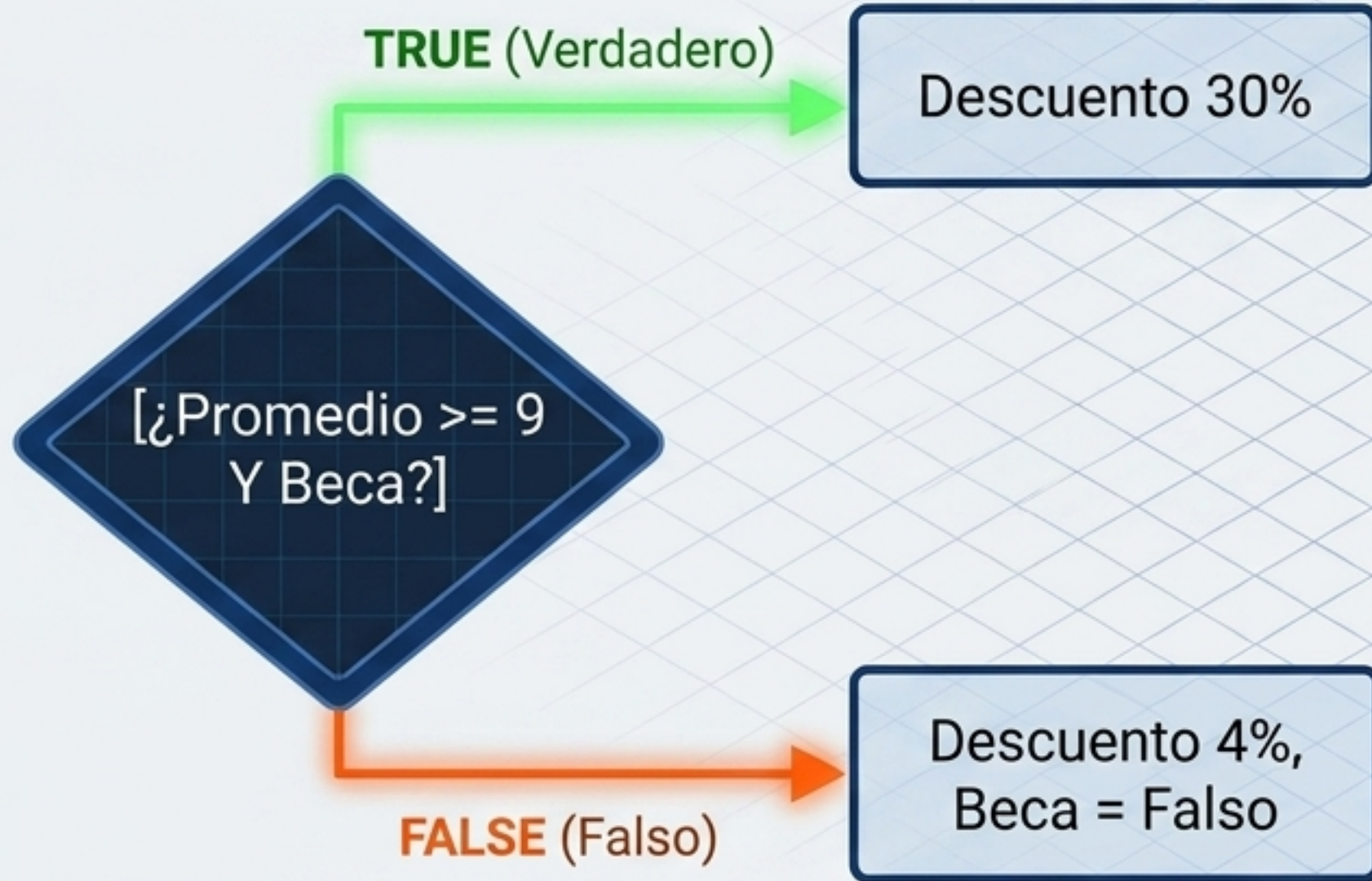
Leer

Pausa la ejecución y espera datos del teclado para guardarlos en una variable.

El truco Sin Saltar

Instruye a PSeInt a no generar un salto de línea (enter) después del texto, ideal para solicitar datos de forma estética en la misma línea.

Dirigiendo el Flujo: Condicionales



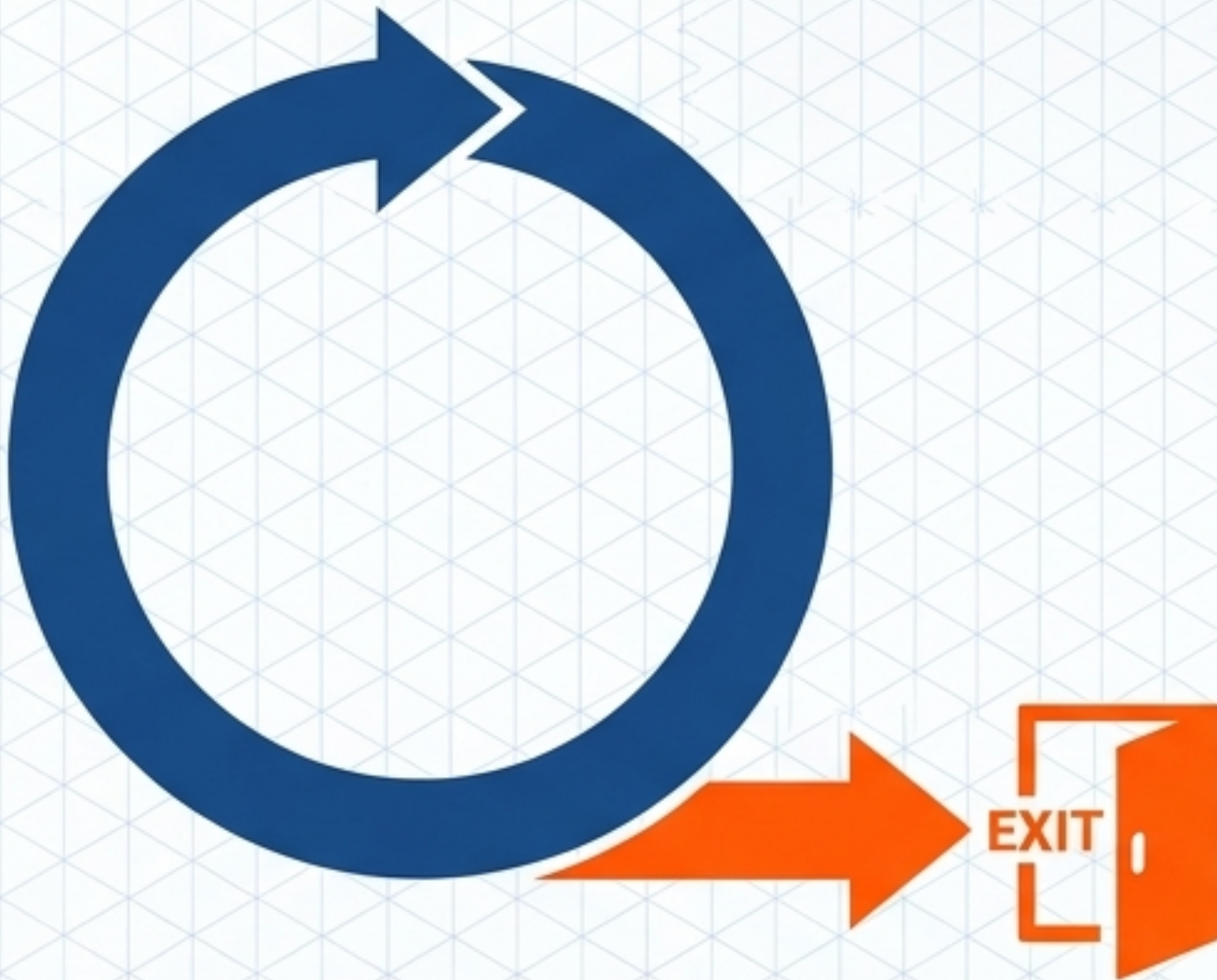
PSeInt Code

```
Si promedio >= 9 Y beca = VERDADERO Entonces  
    descuento = reinscripcion * 0.30  
Sino  
    descuento = reinscripcion * 0.04  
    beca = FALSO  
FinSi
```



Para múltiples opciones cerradas (ej. menús), utilice la estructura **Según** (variable) **Hacer ...**
Caso 1: ... FinSegún.

Dirigiendo el Flujo: Ciclos (Bucles)



```
registrarVenta = "si"  
Mientras registrarVenta = "si" Hacer  
    // ... [Proceso de venta] ...  
    Escribir "¿Desea agregar otra venta? (si/no)"  
    Leer registrarVenta  
FinMientras
```

La Condición de Salida

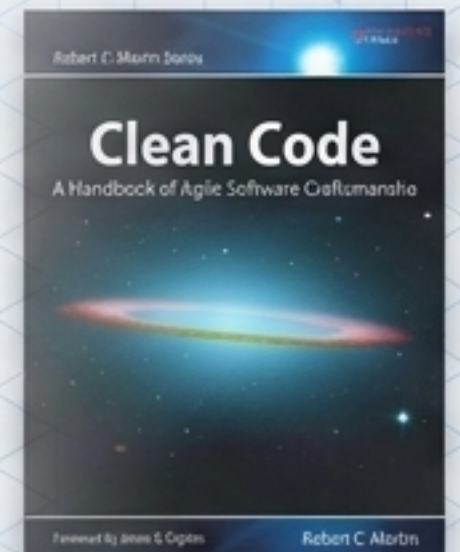
Todo ciclo Mientras o Repetir Hasta Que necesita una actualización de la variable evaluada dentro del ciclo (ej. Leer registrarVenta). Omitir esto genera un Bucle Infinito, bloqueando el programa.

La Maestría: Clean Code

Cualquier tonto puede escribir código que una computadora pueda entender. Los buenos programadores escriben código que los humanos pueden entender.

— Robert C. Martin ("Uncle Bob")

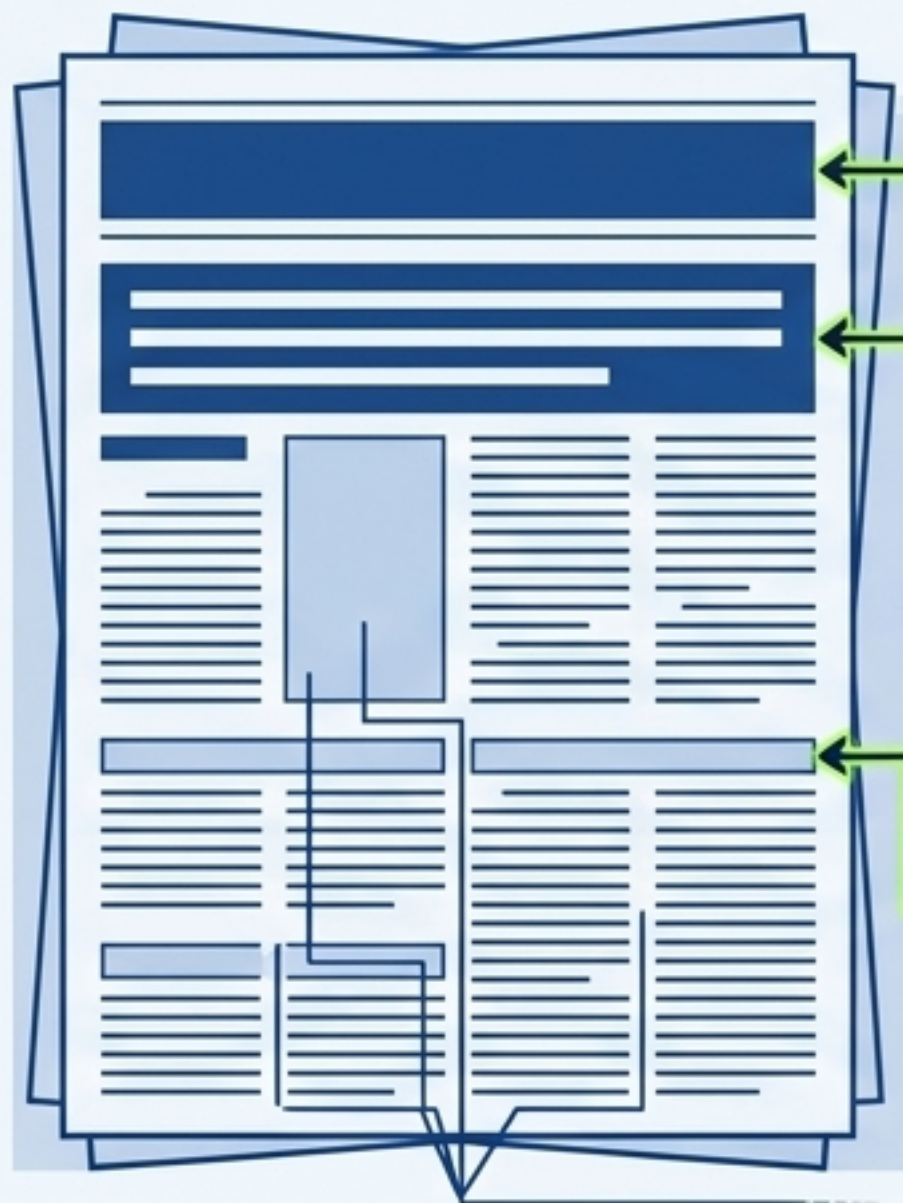
Escribir software no es un acto mecánico; es una disciplina de artesanía (craftsmanship). Un código funcional pero sucio es deuda técnica pura.



La Maestría: Clean Code - Do This / Not That

Código Sucio	Código Limpio
Nombres Genéricos / Abreviaturas: var1, tmp, n, p	Nombres Descriptivos: nombreEmpleado, promedioVentas
Inconsistencia: Nombre_empleado, EDADUSUARIO, Tel_num	Convenciones Estándar: Usar camelCase (ej. precioFinal) o snake_case (ej. precio_final) de forma rigurosa.
Comentarios Redundantes: // Variable para la edad Definir edad Como Entero	El código se explica solo: Las variables bien nombradas hacen que los comentarios sean innecesarios.
Números Mágicos: impuesto = subtotal * 0.21	Variables Explicativas: IVA = 0.21 impuesto = subtotal * IVA

Formato y Estructura (La Metáfora del Periódico)



El código debe leerse como un buen artículo.

1. **Titular:** El nombre del algoritmo indica su propósito general.
2. **Resumen:** Las variables al inicio presentan los componentes principales.
3. **Detalles:** La lógica más densa y los bucles descienden de manera secuencial.

La Verdad sobre los Comentarios



Comentar código malo para intentar explicarlo.



Reescribir el código malo para que sea evidente.



Comentarios de bitácora (ej. 'Modificado por Juan').



Comentarios de advertencia (ej. 'No cambiar este índice, el servidor fallará').

Regla de Oro: Los comentarios no compensan el mal código. Escribe código que se explique a sí mismo.

Método: Estructura impecable E-P-S; las variables se definen primero.

Maestría (Clean Code): Variables descriptivas en camelCase, sin 'números mágicos' ni nombres genéricos.

Proceso ValidarAcceso

```
Definir intentos Como Entero
Definir claveCorrecta Como Lógico
intentos <- 0
claveCorrecta <- FALSO
Mientras intentos < 3 Y claveCorrecta = FALSO Hacer
  Escribir "Ingrese PIN:" Sin Saltar
  // ... (lógica de validación)
FinMientras
FinProceso
```

Ejecución: Uso eficiente del bucle **Mientras** con una condición de salida doble perfecta.

Mentalidad: Abstracción y Descomposición aplicadas: un problema de seguridad real reducido a 10 líneas lógicas.

El código es tu plano definitivo. Diseñalo con propósito.